PART 2 — COMPLETE JAVASCRIPT MASTER NOTES

JavaScript Basic → Advanced | Syntax • DOM • ES6+ • Async • APIs • Browser Programming

Detailed standalone handbook: Basic → Intermediate → Advanced, with syntax, complete examples, expected output/behavior, debugging, projects, interview and viva preparation.

ROADMAP

- JavaScript runtime model
- Variables/types/operators
- Control flow/functions
- Arrays/objects/strings
- DOM/events/forms
- ES6+ and modules
- OOP/prototypes/classes
- Promises/async-await/event loop
- Fetch/REST/JSON
- Storage/browser APIs
- Errors/debugging
- Performance/security
- Projects
- Interview/viva

1. JavaScript Fundamentals

JavaScript is a programming language used in browsers, servers and many other runtimes. In frontend development it controls behavior, data transformation and communication with APIs.

Variables: let, const, var

```
let name = "Aryan";
const age = 21;
var legacy = "avoid in modern code";

name = "Developer";
console.log(name, age);
```

Primitive data types

Type	Example
string	"hello"
number	42, 3.14, NaN, Infinity
bigint	123n
boolean	true / false
undefined	let x;
null	null
symbol	Symbol("id")

Objects and arrays

```
const user = {
  name: "Aryan",
  skills: ["HTML", "CSS", "JS"]
};

const marks = [90, 85, 92];
console.log(user.skills[0]);
```

Operators

```
// arithmetic
+ - * / % **

// comparison
=== !== > < >= <=

// logical
&& || !

// nullish
??

// optional chaining
user.address?.city
```

Key point: Prefer const by default. Use let when reassignment is required. Avoid var in modern application code.

2. Control Flow + Functions

Conditions and loops control program execution. Functions encapsulate reusable logic.

if / else

```
const score = 82;

if (score >= 90) {
  console.log("A");
} else if (score >= 75) {
```

```
  console.log("B");
} else {
  console.log("C");
}
```

switch

```
switch (role) {
  case "admin":
    console.log("Full access");
    break;
  case "student":
    console.log("Student access");
    break;
  default:
    console.log("Guest");
}
```

Loops

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}

for (const skill of skills) {
  console.log(skill);
}

let i = 0;
while (i < 5) {
  i++;
}
```

Functions

```
function add(a, b) {
  return a + b;
}

const multiply = (a, b) => a * b;

const greet = name => `Hello ${name}`;

console.log(add(2, 3));
```

Default/rest/spread

```
function greet(name = "Guest") {
  return `Hi ${name}`;
}

function sum(...nums) {
  return nums.reduce((a,b) => a+b, 0);
}

const a = [1,2];
const b = [...a, 3, 4];
```

3. Strings, Arrays and Objects

Modern JavaScript provides powerful methods for data processing.

String methods

```
const text = " Full Stack JS ";

text.trim();
text.toUpperCase();
text.toLowerCase();
text.includes("Stack");
text.startsWith(" Full");
text.replace("JS", "JavaScript");
text.split(" ");
```

Array methods

```
const nums = [1,2,3,4,5];

nums.map(n => n * 2);
nums.filter(n => n % 2 === 0);
nums.find(n => n > 3);
nums.some(n => n > 4);
nums.every(n => n > 0);
nums.reduce((sum,n) => sum+n, 0);
nums.includes(3);
```

Destructuring

```
const [first, second] = nums;

const user = {name:"Aryan", age:21};
const {name, age} = user;
```

Object utilities

```
Object.keys(user);
Object.values(user);
Object.entries(user);

const copy = {...user};
const merged = {...user, role:"developer"};
```

Key point: map transforms, filter selects, find returns one matching item, reduce accumulates, some checks whether any item matches, every checks whether all match.

4. DOM and Browser Programming

The DOM represents the HTML document as objects. JavaScript can query, modify and create nodes.

Selecting elements

```
document.getElementById("title");
document.querySelector(".card");
document.querySelectorAll(".item");
```

Changing content and attributes

```
const title = document.querySelector("#title");

title.textContent = "New title";
title.innerHTML = "<strong>Bold</strong>";
title.setAttribute("data-id", "42");
title.classList.add("active");
title.classList.toggle("hidden");
```

Creating elements

```
const li = document.createElement("li");
li.textContent = "React";
document.querySelector("ul").append(li);
```

Events

```
const button = document.querySelector("#save");

button.addEventListener("click", event => {
  console.log("Clicked", event);
});
```

Form events

```
form.addEventListener("submit", e => {
  e.preventDefault();

  const data = new FormData(form);
  console.log(data.get("email"));
});
```

Event delegation

```
list.addEventListener("click", e => {
  const button = e.target.closest(".delete");

  if (!button) return;

  console.log(button.dataset.id);
});
```

5. Modern JavaScript / ES6+

ES6 introduced let/const, arrow functions, classes, modules, destructuring, template literals and more. Modern JavaScript continues to evolve.

Template literals

```
const name = "Aryan";
const message = `Hello ${name}!`;
```

Classes

```
class User {
  constructor(name) {
    this.name = name;
  }

  greet() {
    return `Hi ${this.name}`;
  }
}

const u = new User("Aryan");
console.log(u.greet());
```

Inheritance

```
class Admin extends User {
  constructor(name) {
    super(name);
    this.role = "admin";
  }
}
```

Modules

```
// math.js
export function add(a,b) {
  return a+b;
}

// app.js
import { add } from "./math.js";
console.log(add(2,3));
```

Optional chaining and nullish coalescing

```
const city = user.address?.city ?? "Unknown";
```

6. Asynchronous JavaScript

Async programming is essential for API calls, timers, files and other operations that should not block the main thread.

Promise

```
const promise = fetch("/api/users");

promise
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

async / await

```
async function loadUsers() {
```

```
  try {
    const response = await fetch("/api/users");

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }

    const users = await response.json();
    console.log(users);
  } catch (error) {
    console.error(error);
  }
}
```

Promise.all

```
const [users, posts] = await Promise.all([
  fetch("/api/users").then(r => r.json()),
  fetch("/api/posts").then(r => r.json())
]);
```

> **Key point:** Always handle rejected promises. Check response.ok when using fetch because HTTP error statuses do not automatically reject the fetch promise.

7. Fetch API, JSON and Storage

Frontend applications commonly consume REST APIs and persist small amounts of client-side state.

GET

```
const response = await fetch("/api/students");
const students = await response.json();
```

POST JSON

```
await fetch("/api/students", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    name: "Aryan",
    age: 21
  })
});
```

localStorage

```
localStorage.setItem("theme", "dark");

const theme = localStorage.getItem("theme");

localStorage.removeItem("theme");
localStorage.clear();
```

Store objects

```
localStorage.setItem(
  "user",
  JSON.stringify(user)
);

const saved = JSON.parse(
  localStorage.getItem("user")
);
```

8. Advanced JavaScript

Closures, scope, this, prototypes, higher-order functions, error handling and performance form the advanced foundation.

Scope and closure

```
function counter() {
```

```
  let count = 0;

  return function () {
    count++;
    return count;
  };
}

const next = counter();

console.log(next()); // 1
console.log(next()); // 2
```

this

```
const user = {
  name: "Aryan",
  greet() {
    console.log(this.name);
  }
};

user.greet();
```

Error handling

```
try {
  riskyOperation();
} catch (error) {
  console.error(error.message);
} finally {
  console.log("cleanup");
}

throw new Error("Invalid input");
```

Custom error

```
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = "ValidationError";
  }
}
```

Debounce

```
function debounce(fn, delay) {
  let timer;

  return (...args) => {
    clearTimeout(timer);

    timer = setTimeout(() => {
      fn(...args);
    }, delay);
  };
}
```

Key point: Understand closures and the event loop before attempting advanced framework optimization. They explain much of what happens under the hood in frontend applications.

9. Browser APIs and Advanced Topics

Important browser-side APIs include URL/URLSearchParams, History, timers, IntersectionObserver, AbortController and Clipboard.

AbortController

```
const controller = new AbortController();

fetch("/api/data", {
  signal: controller.signal
});

// cancel
```

```
controller.abort();
```

IntersectionObserver

```
const observer = new IntersectionObserver(entries => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      console.log("Visible");
    }
  });
});

observer.observe(document.querySelector(".card"));
```

URLSearchParams

```
const params = new URLSearchParams({
  page: "2",
  search: "react"
});

fetch(`/api/products?${params}`);
```

History API

```
history.pushState(
  {page: 2},
  "",
  "/products?page=2"
);
```

10. Debugging, Security and Best Practices

Use strict input handling, safe DOM APIs and browser tooling.

Debugging workflow

• Use Console for runtime errors.

• Use Sources/Debugger for breakpoints.

• Use Network for API and asset failures.

• Use Elements for DOM inspection.

• Check HTTP status and response payload.

• Reduce complex bugs to a minimal reproducible case.

Avoid unsafe HTML injection

```
// Prefer:
element.textContent = userInput;

// Be careful with:
element.innerHTML = userInput;
```

Key point: Never assume client-side JavaScript can protect secrets. API keys, authorization decisions and sensitive validation belong on trusted backend systems.

PRACTICAL PROJECTS

Project 1 — Todo App

DOM + events + localStorage.

```
const form = document.querySelector("#todoForm");
const input = document.querySelector("#todoInput");
const list = document.querySelector("#todoList");

form.addEventListener("submit", e => {
  e.preventDefault();
  if (!input.value.trim()) return;
```

```
  const li = document.createElement("li");
  li.textContent = input.value.trim();

  const remove = document.createElement("button");
  remove.textContent = "Delete";
  remove.addEventListener("click", () => li.remove());

  li.append(" ", remove);
  list.append(li);
  input.value = "";
});
```

Expected result/preview: run the example in the stated environment. The visible result should match the described component or behavior. Use DevTools/console/network tools to verify it.

Project 2 — API Search

Fetch data, render results, handle errors.

```
async function searchUsers(query) {
  const response = await fetch(
    `/api/users?search=${encodeURIComponent(query)}`
  );

  if (!response.ok) {
    throw new Error("Request failed");
  }

  return response.json();
}
```

Expected result/preview: run the example in the stated environment. The visible result should match the described component or behavior. Use DevTools/console/network tools to verify it.

Project 3 — Form Validation

Use HTML validation plus JavaScript for custom feedback.

```
form.addEventListener("submit", e => {
  if (!form.checkValidity()) {
    e.preventDefault();
    form.reportValidity();
  }
});
```

Expected result/preview: run the example in the stated environment. The visible result should match the described component or behavior. Use DevTools/console/network tools to verify it.

INTERVIEW + VIVA REVISION

• What is the difference between let, const and var?

• Primitive vs reference values?

• == vs ===?

• What is hoisting?

• What is a closure?

• What is lexical scope?

• How does this work?

• What is the prototype chain?

• Class vs prototype?

• What is an event loop?

• Microtask vs task queue?

• Promise states?

• async/await?

• Why check response.ok?

• map vs forEach?

• filter vs find?

• reduce?

• Shallow vs deep copy?

• Spread vs rest?

• Optional chaining?

• Nullish coalescing?

• Event bubbling/delegation?

• preventDefault vs stopPropagation?

• localStorage vs sessionStorage?

• What is CORS conceptually?

• How do you prevent XSS in DOM code?

• What is debouncing?

• What is throttling?

• What is AbortController?

• How do modules work?

FINAL CHEAT SHEET

Revise the syntax patterns, lifecycle/flow, common errors, and project architecture. The goal is not memorizing every property or method, but knowing the model well enough to build and debug applications.